Lab 6

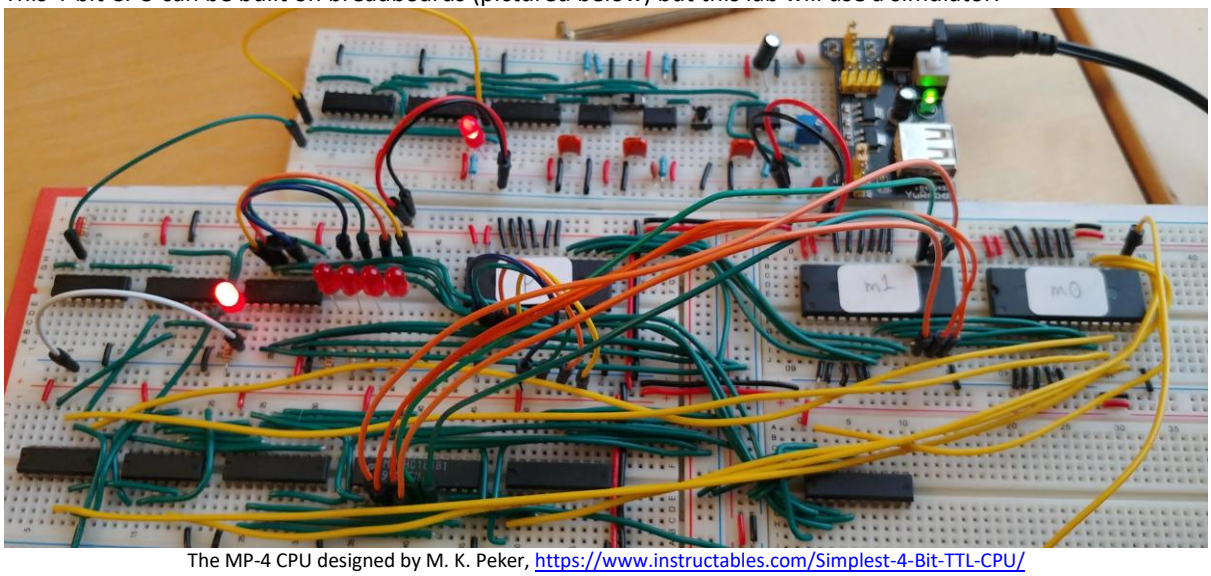
Objective: Program a 4-bit CPU simulator

This lab builds on lectures 5 and 6

Background

In this lab we're going to learn the assembly language of a simple 4-bit CPU, write some programs, and hand assemble them into machine code.

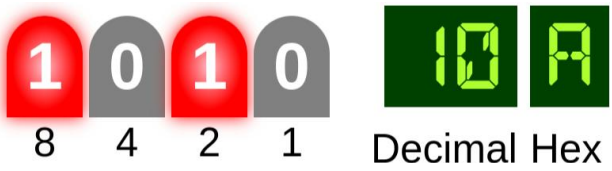
This 4-bit CPU can be built on breadboards (pictured below) but this lab will use a simulator.



The MP-4 CPU designed by M. K. Peker, https://www.instructables.com/Simplest-4-Bit-TTL-CPU/

Exercise 1: Bits and Bytes

We must first understand binary, and hexadecimal encoding.
Below is the number *ten*, as binary-digits: `1010`. Decimal-digits: `10`. Hex-digit: `$A`.



Convert the following binary digits (bits) into hex.

```
1100 1010 1111 1110 1111 0000 0000 1101
```

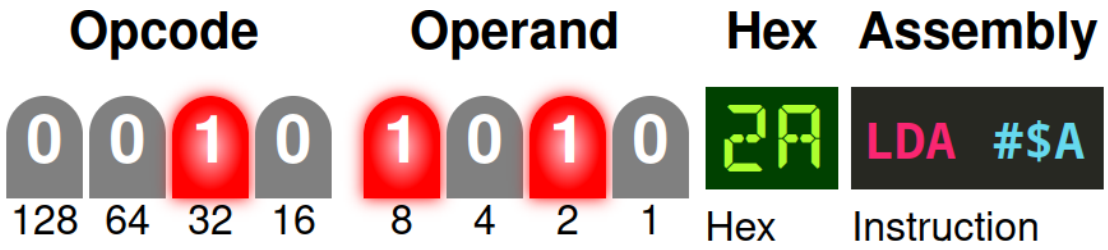
Hints:

Use the **Interactive Nibble** here https://cosc204.cspages.otago.ac.nz/4-bit-cpu/
Each nybble (4-bits) can be considered independently.
The second nybble, is `1010`, which is `$A` in hex.

For the rest of this lab assume numbers are written in hex or binary (no more decimal).

Exercise 2: Assembly Instructions

The instructions for our 4-bit CPU are written in machine code. Each instruction is 1 byte (8 bits). The first 4 bits are the opcode, and the last 4 bits are the operand.



The above byte **`0010 1010`** corresponds to the hex-digits **`$2A`**. Using the table below we can decode **`$2A`** as **`LDA #$A`** which means: load the accumulator with the value **`$A`** (*ten*).

Instruction	Description	Opcode	Operand
`STOP`	Halts the CPU.	`$0`	
`LDA #$v`	Load the **operand** v into the accumulator.	`$2`	`v: $0..$F`
`ADDA #$v`	Add the **operand** v to the accumulator.	`$4`	`v: $0..$F`
`DECA`	Decrement the accumulator by one.	`$6`	
`IN`	Read keypad value and load into the accumulator.	`$8`	
`OUT`	Print the value of the accumulator.	`$A`	
`BRA $a`	Branch to the given **operand** as a memory address.	`$C`	`a: $0..$F`
`BEQ $a`	Branch to **operand** if the accumulator is zero.	`$E`	`a: $0..$F`

$ means *hex* format.
means *immediate* value.

The above instructions are enough to write many programs using our 4-bit CPU. Below is a simple program that prints numbers **`$F`** to **`$1`** (i.e. *fifteen* to *one*). The left pseudocode uses a **`do-while`** loop **.** The right pseudocode instead uses **`labels`** and **`goto`** statements.

Read both and convince yourself that they perform the same computation.

```
Pseudocode
A = $F
do
  print A
  A = A - 1
while (A != 0)
exit
```

Labels	Pseudocode
	`A = $F`
`LOOP:`	`print A`
	`A = A - 1`
	`if (A == 0) goto END`
	`goto LOOP`
`END:`	`exit`

The right pseudocode easily translates into our assembly instructions. Follow it through by hand and confirm that you understand what each instruction does.

Labels	Assembly	Pseudocode
	`LDA #$F`	`A = $F`
`LOOP:`	`OUT`	`print A`
	`DECA`	`A = A - 1`
	`BEQ end`	`if (A == 0) goto END`
	`BRA loop`	`goto LOOP`
`END:`	`STOP`	`exit`

Exercise 3: Turn the program into machine code

Convert the assembly program into machine code. Recall, each assembly instruction is 1-byte. The top nybble is the opcode and the bottom nybble is the operand (if any, if not then $0).

Address	Label	Assembly	Machine Code (Hex)
$00		LDA #$F	
$01	loop:	OUT	
$02		DECA	
$03		BEQ end	
$04		BRA loop	
$05	end:	STOP	$00

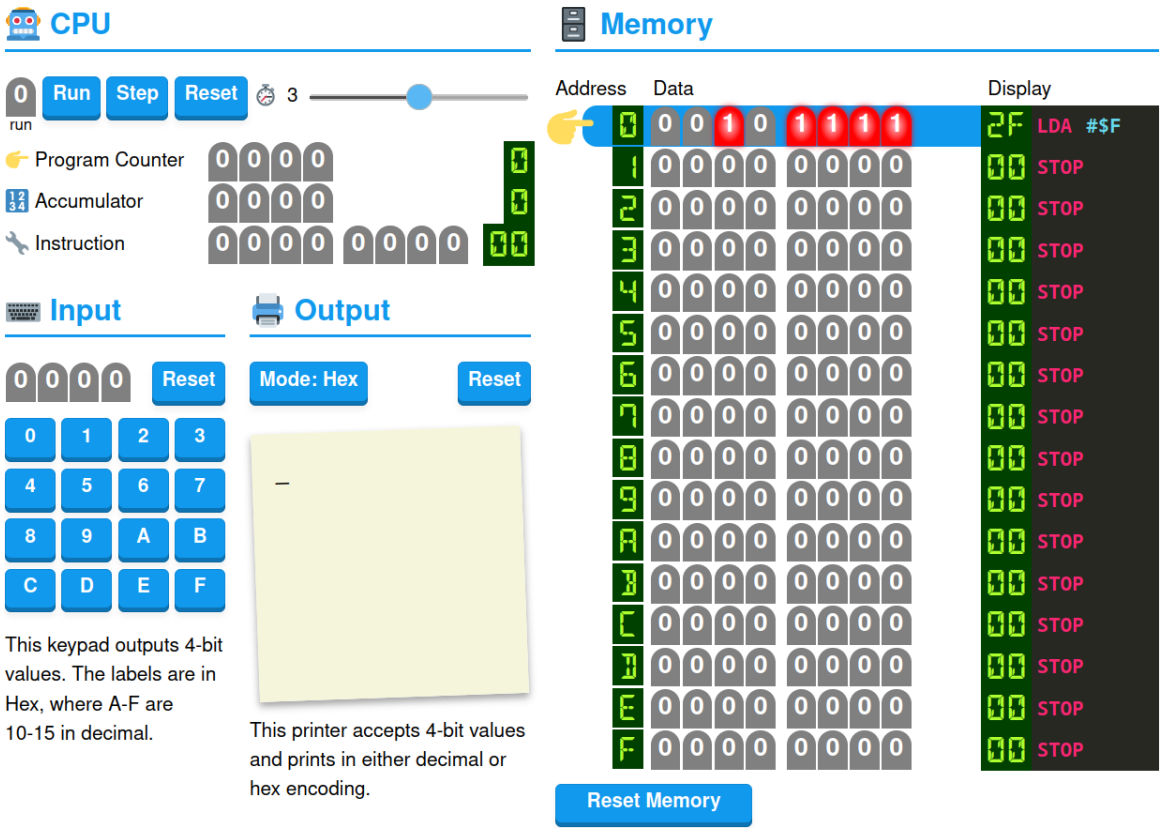
Translate the assembly into machine code by filling in the last column in the table above.

Hints:

LDA #$F uses the opcode **$2** and the operand **$F** to become **$2F**
OUT becomes **$A0** since **$A** is **OUT** and the operand **$0** is ignored.
To translate an instruction with labels, use the line number (address).
For example, on line **$03** the instruction **BEQ end** gets translated into **$E5**
That is, branch to address **$5** if the accumulator is zero.

Now, program your machine code into the online simulator to test it out!
https://cosc204.cspages.otago.ac.nz/4-bit-cpu/

The first instruction **LDA #$F** should appear as below



Exercise 4: More Programs

1. Write a program that counts up, printing from **$1** to **$F** (*one* to *fifteen*).
2. Write a program that counts down, then counts up.
3. Write a program that reads from the keyboard and prints to the output.

Exercise 5: Extending the CPU Simulator

We also have implementations of the 4-bit CPU simulator in Java and C (available on Aoroa), which you can extend with additional instructions. Start with the Java one, as that language will be more familiar to most of you. Download both source files from Aoroa and save them to your J: drive. Read the code (and comments) from top to bottom to understand what each part of the CPU does, and how each instruction is simulated.

As an overview: the program reads a byte from `memory[pc]`, decodes it into the opcode and operand, `switch`es on the opcode then carries out the instruction on the operand.

You may want to add some debugging output (`System.err.println()` in Java, `printf()` in C). The obvious place to do this is just before the line that reads "`pc++;`". If you print `pc`, the accumulator, and all 16 bytes of memory, then it will give you the entire state of the computer before it executes each instruction. You might also want to print `opcode` and `operand` at this point too.

Compiling and Running the Java-based Simulator

Open the folder containing the source files in Windows Terminal and run the following commands to first compile and then run the program. Note: because the memory is initially empty, the program will produce no output! (In Exercise 6 you will enter programs.)

```
javac FourBitCPU.java
java FourBitCPU
```

Compiling and Running the C-based Simulator

To compile the C-based simulator, type:

```
gcc -Wall -Wextra 4-bit-CPU.c -o 4-bit-CPU.exe
```

To run:

```
./4-bit-CPU.exe
```

Exercise 6: Simulate!

You have your machine code program(s) from Exercises 3 and 4. For each program, enter the program data in the line that initialises `memory`, compile and run the simulator and check the output.

Exercise 7: New instructions to read and write memory

In this exercise we're going to add two new instructions:

Instruction	Description	Opcode	Operand
`STA $m`	Store the accumulator at **memory address** m	`$1`	`m: $0..$F`
`LDA $m`	Load the accumulator with the value at **memory address** m	`$3`	`m: $0..$F`

Note the difference between **LDA #$v** and **LDA $m**. The '**#**' means "with the value", when it is not present it means "from memory location". This is important to understand when doing your assignment.

We need to assign opcodes to the instructions:

Opcode	Instruction
$1	STA
$3	LDA

Add constants for these that we can use in the `switch` statement. To distinguish memory-based read/write from value-based read/write we'll put an "`_M`" on the end of the constant to signify "from memory".

For the Java implementation:

```
public static final int STA_M = 0x10;
public static final int LDA_M = 0x30;
```

For the C implementation:

```
#define STA_M    0x10
#define LDA_M    0x30
```

Now add code to the switch statement to implement the new instructions, and write a program to test them.

Exercise 8: Fun!

Your new simulator will run your old programs as before. You added new instructions while remaining binary backwards compatible with the old version (just like ARM and Intel do!). But we can now write new programs. Let's start with a program that, in a loop, reads a hex digit from the terminal and prints out the number of times we've seen a zero.

Address	4-bit	Machine Code
more: $00 $01 $02	more: IN BEQ add_one BRA print	
add_one: $03 $04 $05	add_one: LDA store ADDA #$1 STA store	
print: $06 $07 $08	print: LDA store OUT BRA more	
store: $09	store: $00	

Have some fun! Write some programs that do cool stuff. If you need a new instruction, then add it. You could add a compare instruction – it might compare the accumulator with a value and set the value of the accumulator to $0 for equal, $1 for greater, or $F for less than. With `CMP #$v` you might be able to write a "guess the number" game (albeit with a somewhat restricted numeric range to guess from, and with the value hard-coded into the program).